

Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

Entrega Final — Documento Consolidado

Período: **08/05/2026** a **03/07/2026**

Em grupo | Todos os grupos abordam os 5 temas | 8 sprints + entrega final

Documento Final Consolidado

Relatório técnico — compilação revisada das entregas das Sprints 1 a 7

Integrantes

Gabriel Henrique
Clara Catarina
Ruan Cabral
Lucas Nunes

Análise e Desenvolvimento de Sistemas · Gerenciamento Ágil de Projetos
Rio de Janeiro — 03 de julho de 2026

Sumário

1. Introdução.....	3
2. Design Thinking aplicado a software.....	3
3. Lean e Agilidade.....	3
4. Kanban: visualizando o trabalho.....	4
5. Extreme Programming (XP).....	5
6. DDD: entendendo o domínio do problema.....	6
7. Integração entre os temas.....	7
8. Conclusão.....	7
Referências.....	7

1. Introdução

Este relatório consolida as entregas produzidas pelo grupo ao longo das oito sprints da disciplina de Gerenciamento Ágil de Projetos. O trabalho aborda cinco temas centrais do ecossistema ágil — Design Thinking, Lean e Agilidade, Kanban, Extreme Programming (XP) e Domain-Driven Design (DDD) — apresentando, para cada um, seus fundamentos, sua aplicação prática e uma análise crítica.

Como fio condutor, utilizamos o projeto **Scrunic**, uma plataforma de gestão de tarefas com quadro Kanban desenvolvida pelo grupo com Node.js e MySQL no back-end e React com TypeScript no front-end. Os conceitos estudados foram aplicados diretamente nesse projeto, o que permitiu observar na prática como as abordagens ágeis se complementam — tema aprofundado na seção de integração.

2. Design Thinking aplicado a software

O Design Thinking é uma abordagem centrada no ser humano para a resolução de problemas. Em vez de partir da solução, parte-se do entendimento profundo de quem usará o produto e de qual dor precisa ser resolvida. No desenvolvimento de software, ele atua principalmente na fase de descoberta, reduzindo o risco de construir um sistema tecnicamente correto, porém inútil para o usuário.

2.1 As cinco etapas

- **Empatia.** Imersão no contexto do usuário por meio de entrevistas e observação, para compreender necessidades reais e não supostas.
- **Definição.** Síntese das descobertas em um enunciado claro do problema a resolver.
- **Ideação.** Geração ampla de ideias de solução, sem julgamento prévio, seguida de seleção das mais promissoras.
- **Prototipação.** Construção de versões simples e baratas da solução para materializar as ideias.
- **Teste.** Validação dos protótipos com usuários reais, gerando aprendizado que realimenta as etapas anteriores.

2.2 Aplicação no projeto Scrunic

No Scrunic, a etapa de empatia revelou que os usuários sentiam falta, acima de tudo, de enxergar o próprio trabalho: tarefas espalhadas em anotações e mensagens dificultavam saber o que estava em andamento. A definição do problema — “falta de visibilidade do fluxo de trabalho” — direcionou a ideação para soluções visuais, e o protótipo de um quadro estilo Kanban foi o mais bem avaliado nos testes. Essa descoberta definiu a própria natureza do produto e conectou o Design Thinking aos demais temas do trabalho.

Síntese crítica: o Design Thinking garante que se construa a coisa certa antes de construí-la do jeito certo. Sua limitação é não prescrever como gerenciar a construção em si — papel dos métodos ágeis apresentados a seguir.

3. Lean e Agilidade

O pensamento Lean nasceu no Sistema Toyota de Produção e foi adaptado ao software por Mary e Tom Poppendieck. Seu objetivo é maximizar o valor entregue ao cliente eliminando desperdícios — tudo aquilo que consome recursos sem agregar valor. A agilidade, formalizada no Manifesto Ágil (2001), materializa esses princípios no dia a dia do desenvolvimento.

3.1 Princípios do Lean aplicados ao software

- **Eliminar desperdício.** Cortar funcionalidades supérfluas, retrabalho, esperas e burocracia que não agregam valor.
- **Amplificar o aprendizado.** Trabalhar em ciclos curtos com feedback constante, tratando o desenvolvimento como processo de descoberta.

- **Decidir o mais tarde possível.** Adiar decisões irreversíveis até que haja informação suficiente.
- **Entregar o mais rápido possível.** Fluxo contínuo de pequenas entregas, encurtando o tempo entre pedido e valor.
- **Empoderar a equipe.** Quem faz o trabalho é quem melhor sabe como fazê-lo; a gestão dá suporte, não microgerencia.
- **Construir com qualidade.** Qualidade embutida no processo (testes, integração), não inspecionada apenas ao final.
- **Otimizar o todo.** Olhar para o fluxo de valor completo, e não para a eficiência isolada de cada etapa.

3.2 Do Lean à agilidade

O Manifesto Ágil traduz o espírito Lean em quatro valores: indivíduos e interações mais que processos e ferramentas; software funcionando mais que documentação abrangente; colaboração com o cliente mais que negociação de contratos; e resposta a mudanças mais que seguir um plano. No Scrunic, esses valores orientaram a decisão de entregar primeiro um produto mínimo viável — login, cadastro e quadro de tarefas — e evoluir a partir do feedback, em vez de tentar construir o sistema completo de uma só vez.

Síntese crítica: o Lean fornece a filosofia (valor e fluxo) e a agilidade fornece os valores de trabalho; ambos, porém, precisam de métodos concretos para operar — e é aí que entram o Kanban e o XP.

4. Kanban: visualizando o trabalho

4.1 Conceitos fundamentais

Visualização do trabalho. O trabalho do conhecimento muitas vezes é invisível. O Kanban resolve isso transformando cada tarefa em um cartão e o processo em colunas, permitindo que toda a equipe veja imediatamente o que está pendente, em andamento e finalizado, garantindo transparência total sobre o status do projeto.

Limites de WIP (trabalho em progresso). É a regra de limitar a quantidade máxima de tarefas simultâneas em cada etapa. Em vez de começar várias coisas e não terminar nenhuma, a equipe foca em concluir o que já começou — o que evita sobrecarga, reduz troca de contexto e ajuda a identificar rapidamente onde o trabalho está travando.

Fluxo. Refere-se à movimentação contínua e previsível das tarefas do pedido à entrega. O Kanban utiliza um sistema “puxado”: uma tarefa só avança quando há espaço disponível na próxima etapa, garantindo que ninguém receba mais trabalho do que consegue processar.

4.2 Como um quadro Kanban opera na prática

Tudo começa na fila de espera, onde as demandas são ordenadas por prioridade. Com capacidade livre, a equipe puxa a tarefa mais importante para a execução, ocupando um espaço no limite de WIP daquela etapa. Se uma coluna atinge seu limite, ninguém puxa novas tarefas: o esforço é direcionado a destravar o gargalo. Concluídas as validações, a tarefa é movida para a coluna final, liberando espaço para o fluxo recomeçar. Um quadro bem-sucedido não tem muitos cartões abertos ao mesmo tempo — tem cartões que cruzam o quadro de forma fluida, controlada e sem interrupções.

4.3 Simulação de fluxo: sistema de agendamento médico

Para exercitar o método, simulamos o desenvolvimento de um sistema de agendamento médico online com o seguinte quadro:

Backlog	A Fazer (WIP: 5)	Em Desenv. (WIP: 3)	Revisão (WIP: 2)	Testes (WIP: 2)	Concluído
Integração com WhatsApp	Tela de login	API de autenticação	Cadastro de consultas	Recuperação de senha	Banco de dados

Backlog	A Fazer (WIP: 5)	Em Desenv. (WIP: 3)	Revisão (WIP: 2)	Testes (WIP: 2)	Concluído
Relatórios gerenciais	Cadastro de pacientes	Tela de agendamento	Especialidades médicas	Autenticação de usuários	Estrutura inicial
Notificações automáticas	Cadastro de médicos; recuperação de senha; menu principal	Regras de negócio de consultas			

4.4 Análise crítica

Vantagens: transparência total do status, redução de sobrecarga pelos limites de WIP, identificação rápida de gargalos, flexibilidade (sem prazos ou papéis fixos) e melhoria contínua guiada por métricas de fluxo (lead time, cycle time e throughput).

Limitações e desafios: a ausência de prazos fixos dificulta previsões; a liberdade do método pode virar desorganização em times iniciantes; sem priorização o backlog vira fila infinita; e todo o benefício depende da disciplina de respeitar os limites de WIP e manter o quadro atualizado.

Comparação com o Scrum: o Kanban trabalha em fluxo contínuo, sem papéis nem cerimônias obrigatórias, e aceita mudanças a qualquer momento; o Scrum trabalha em sprints de tempo fixo, com papéis definidos e escopo protegido durante a iteração. O Kanban rende mais com demanda contínua e imprevisível; o Scrum, com trabalho planejável em ciclos.

5. Extreme Programming (XP)

O Extreme Programming é uma metodologia ágil focada em melhorar a qualidade do software e facilitar adaptações rápidas às mudanças de requisitos, por meio de valores fundamentais e práticas que incentivam colaboração, simplicidade e entregas contínuas.

5.1 Valores do XP

- **Comunicação.** A comunicação constante entre desenvolvedores e clientes evita erros e garante que todos entendam os objetivos do projeto.
- **Simplicidade.** O desenvolvimento foca apenas no necessário no momento, evitando complexidade desnecessária.
- **Feedback.** O retorno rápido permite corrigir problemas e melhorar funcionalidades continuamente.
- **Coragem.** A equipe deve ter coragem para modificar, refatorar e melhorar o código sempre que necessário.
- **Respeito.** Todos os membros respeitam as opiniões e contribuições uns dos outros.

5.2 Práticas do XP

- **TDD (Test-Driven Development).** Os testes são criados antes da implementação do código.
- **Pair Programming.** Dois desenvolvedores trabalham juntos no mesmo computador, um programando e outro revisando.
- **Integração Contínua.** O código é integrado frequentemente ao projeto principal, com testes automáticos executados constantemente.
- **Refatoração.** Melhorar a estrutura do código sem alterar seu funcionamento.
- **Pequenas Releases.** O sistema é entregue em pequenas versões frequentes para receber feedback rapidamente.

5.3 Estudo de caso: três práticas XP no Scrunic

TDD. No módulo de Criação de Tarefas, antes de criar a rota no back-end (Node.js) ou a tabela no MySQL, o desenvolvedor escreveu o teste automatizado `deveRejeitarTarefaSemTitulo()`. O teste falhou (fase vermelha); escreveu-se então apenas o código de validação necessário para passá-lo (fase verde); por fim, o código foi refatorado mantendo a simplicidade sem quebrar a funcionalidade.

Pair Programming. O quadro Kanban interativo do front-end exigia lógica complexa de estados no React. Dois programadores compartilharam o mesmo computador: o “piloto” implementou a interface e o arrastar-e-soltar dos cartões, enquanto o “navegador” revisava o TypeScript em tempo real, sugeria melhorias de arquitetura e antecipava problemas — evitando refatorações futuras e acelerando a entrega.

Integração Contínua. Com vários desenvolvedores atuando nos módulos de login, cadastro e gestão de usuários, a equipe configurou automação no repositório: a cada commit, a aplicação era compilada e toda a suíte de testes do TDD executada. Se algo conflitasse com o banco MySQL ou fizesse um teste falhar, o envio era rejeitado imediatamente, permitindo corrigir o erro na hora.

5.4 Análise crítica

Vantagens: alta qualidade de código (TDD), detecção precoce de erros (integração contínua), conhecimento compartilhado (pareamento), adaptabilidade a mudanças e feedback rápido via pequenas releases. **Limitações:** alta exigência de disciplina, custo inicial do pareamento, dependência da participação ativa do cliente e foco quase exclusivo na engenharia, dizendo pouco sobre gestão e priorização.

XP + Scrum: por atuarem em níveis diferentes — o Scrum na gestão (o quê e quando entregar) e o XP na engenharia (como construir com qualidade) — os dois formam um híbrido natural: o Scrum dá o ritmo com sprints e cerimônias, e o XP garante que cada incremento seja confiável e fácil de evoluir.

6. DDD: entendendo o domínio do problema

6.1 Conceitos estratégicos

Domínio. É a razão de ser do software: a área de negócio, o problema real a resolver. O foco é entender como as regras funcionam no mundo real antes de escrever código. O domínio é dividido em subdomínios (principal, de suporte e genérico) para facilitar o gerenciamento do sistema.

Linguagem Ubíqua. É o idioma comum falado por todos os envolvidos, unindo desenvolvedores e especialistas do negócio. Os mesmos termos das reuniões devem aparecer nos diagramas, nas tarefas e, principalmente, no código-fonte (nomes de classes, funções e variáveis), eliminando falhas de comunicação entre negócio e TI.

Contexto Delimitado. É a fronteira lógica onde um modelo de domínio e sua Linguagem Ubíqua se aplicam. A palavra “Produto”, por exemplo, em Vendas tem preço, estoque e descrição; em Logística, o mesmo “Produto” vira um item com peso, dimensões e código de rastreio. Delimitar contextos evita modelos gigantescos e confusos.

6.2 Padrões táticos: modelando o domínio

Entidades. Possuem identidade única e ciclo de vida: nascem, são modificadas e podem ser destruídas, mas seu ID permanece. A igualdade entre entidades é determinada pelos IDs, não pelos valores; elas contêm comportamento e garantem que suas propriedades permaneçam válidas após qualquer modificação.

Value Objects. Não possuem identidade própria: descrevem, quantificam ou qualificam características de uma entidade. Dois VOs com os mesmos valores são idênticos e intercambiáveis. São imutáveis — para alterar, descarta-se o antigo e cria-se um novo — e validam a si mesmos no momento da criação.

Agregados e Raízes de Agregação. Um agregado é uma fronteira de consistência que agrupa entidades e VOs que precisam mudar juntos. O mundo externo só conversa com a Raiz do Agregado,

que coordena as mudanças internas e garante que as regras de negócio (invariantes) se mantenham ao final de cada transação.

Repositórios. Apenas Raízes de Agregação possuem repositórios. O repositório funciona como uma coleção em memória daquela raiz: carrega o agregado inteiro e o persiste de forma atômica, escondendo do domínio a complexidade de tabelas SQL ou coleções NoSQL.

6.3 Aplicação no Scrunic

No Scrunic, a Linguagem Ubíqua orientou a nomenclatura de classes e tabelas — Tarefa, Quadro, Coluna, Usuário — espelhando exatamente os termos do negócio. A Tarefa foi modelada como entidade (tem ID e ciclo de vida), enquanto atributos descritivos como prioridade se comportam como objetos de valor. O Quadro atua como raiz de agregação das suas colunas e cartões, com um repositório responsável por carregar e persistir o agregado de forma consistente no MySQL.

Síntese crítica: o DDD mantém o código alinhado ao negócio e brilha em domínios complexos; seu custo é a curva de aprendizado e a disciplina de modelagem, que podem ser desproporcionais em sistemas muito simples.

7. Integração entre os temas

Os cinco temas estudados não competem entre si: atuam em camadas complementares do ciclo de um produto de software, como resume o quadro abaixo.

Tema	Camada de atuação	Contribuição para o produto
Design Thinking	Descoberta	Define o problema certo a resolver, a partir das pessoas.
Lean e Agilidade	Filosofia e valores	Orientam a construção sem desperdício, em ciclos curtos.
Kanban	Gestão do fluxo	Torna o trabalho visível e controlável (WIP, gargalos).
XP	Engenharia	Garante a qualidade técnica do código (TDD, par, CI).
DDD	Modelagem	Mantém o código alinhado ao domínio do negócio.

Dois exemplos concretos de integração foram explorados no trabalho. O primeiro é o **Scrumban**, que combina o planejamento e a cadência do Scrum com os limites de WIP e o fluxo contínuo do Kanban, incluindo uma faixa expressa para urgências de produção. O segundo é o híbrido **XP + Scrum**, em que o Scrum organiza o trabalho (sprints, papéis, priorização) e o XP cuida da engenharia (TDD, pareamento, integração contínua e refatoração). No Scrunic, todos esses elementos atuaram juntos: o Design Thinking definiu o produto, o Lean orientou o corte de desperdícios, o Kanban organizou o fluxo, o XP garantiu a qualidade do código e o DDD manteve o modelo fiel ao negócio.

8. Conclusão

Ao longo das oito sprints, o grupo percorreu o ecossistema ágil da descoberta à engenharia. A principal lição é que a agilidade não é uma ferramenta única, e sim um conjunto de práticas complementares: Design Thinking, Lean, Kanban, XP e DDD, juntos, formam um caminho completo — do entendimento do problema à construção de um software de qualidade, alinhado ao negócio e preparado para mudanças. A aplicação prática no projeto Scrunic mostrou que essas abordagens rendem mais quando combinadas com critério, respeitando o contexto, a maturidade da equipe e a natureza do trabalho.

Referências

ANDERSON, D. J. Kanban: Mudança Evolucionária de Sucesso para seu Negócio de Tecnologia. Blue Hole Press, 2011.

BECK, K. Programação Extrema (XP) Explicada: Acolha as Mudanças. 2. ed. Bookman, 2004.

BECK, K. et al. Manifesto para Desenvolvimento Ágil de Software. 2001. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>.

BROWN, T. Design Thinking: Uma Metodologia Poderosa para Decretar o Fim das Velhas Ideias. Alta Books, 2017.

EVANS, E. Domain-Driven Design: Atacando as Complexidades no Coração do Software. Alta Books, 2016.

POPPENDIECK, M.; POPPENDIECK, T. Lean Software Development: An Agile Toolkit. Addison-Wesley, 2003.

SCHWABER, K.; SUTHERLAND, J. O Guia do Scrum. 2020. Disponível em: <https://scrumguides.org>.

VERNON, V. Implementing Domain-Driven Design. Addison-Wesley, 2013.